

쇼츠 만들기

방배동 예심교회 주일설교 → 세로 쇼츠 3편. 아무것도 없는 새 컴퓨터에서 시작해, 영상이 나오고, 자막까지 고치는 데까지. 전부 무료고 API 키가 필요 없다.

윈도우 10 또는 11 · 64비트

여유 공간 20GB

인터넷

1부 · 설치

Git for Windows
Python (PATH 체크)
Node.js · Claude Code
프로그램 받기
ffmpeg · yt-dlp · whisper
확인 (doctor · 렌더 테스트)

2부 · 만들기

영상 고르기 (플랜 A / B)
다운로드 · 전사
구간 선별 · 렌더
엔드카드 자동 생성
결과 확인 · 업로드

3부 · 고치기

자막 글자 고치기
고친 뒤 다시 렌더
구간 · 설교 범위 조정
엔드카드 수정

1

Git for Windows

git-scm.com/download/win (<https://git-scm.com/download/win>) 에서 받아 실행한다. 선택지가 많이 나오는데 전부 기본값 그대로 **Next** 면 된다.

딱 하나만 확인한다 — `Adjusting your PATH` 화면에서 **“Git from the command line and also from 3rd-party software”**. 그게 기본값이다.

설치가 끝나면 시작 메뉴에서 **Git Bash** 를 찾아 연다.

```
사용자@PC MINGW64 ~  
$
```

일반 명령 프롬프트가 아니다

`명령 프롬프트` 나 `PowerShell` 이 아니라 **Git Bash** 를 쓴다. 앞으로 모든 작업이 이 검은 창에서 이뤄진다.

붙여넣기는 `Ctrl + V` 또는 마우스 오른쪽 버튼 → `Paste`. 회색 상자 안의 것만 복사한다.

```
$ git --version
```

2

Python

[python.org](https://www.python.org/downloads/windows/) (<https://www.python.org/downloads/windows/>) 에서 최신 3.x를 받는다.

첫 화면의 체크박스가 전부다

맨 아래 **Add python.exe to PATH** 를 반드시 체크하고 `Install Now`. 이걸 놓치면 뒤가 전부 안 된다.

Git Bash를 닫고 새로 연 뒤 확인한다.

```
$ python --version
```

버전이 안 나오면 PATH 체크를 놓친 것이다. 다시 설치하면서 그 칸을 체크한다.

3

Node.js와 Claude Code

nodejs.org (<https://nodejs.org>) 에서 **LTS** 를 받아 설치한다. 전부 기본값. 끝나면 Git Bash를 닫고 새로 연다.

```
$ npm install -g @anthropic-ai/claude-code
```

```
$ claude
```

브라우저가 열리면 Claude 계정으로 로그인한다. 끝나면 `/exit` 로 나온다.

추가 비용 없음

Claude 구독이 있으면 그대로 쓴다. 쇼츠 구간을 고르는 데만 사용한다.

4

프로그램 받기

```
$ cd ~
```

```
$ git clone -b claude/shorts-creation-automation-0xaii8 https://github.com/sunki2024-star/shorts-factory
```

```
$ cd ~/shorts-factory
```

`~` 는 `C:\Users\내이름` 을 뜻한다. 파일 탐색기로 찾으려면 거기 있다.

윈도우 사용자 이름이 한글이면

경로에 한글이 있으면 문제가 생길 수 있다. `cd /c/` 로 옮긴 뒤 clone 하고, 그다음부터 `cd /c/shorts-factory` 로 들어간다.

도구 설치

```
$ bash scripts/setup_render_env.sh --with-whisper
```

20~40분 걸린다. 영상 편집기(ffmpeg), 유튜브 다운로더(yt-dlp), 한국어 음성 인식 프로그램과 모델(3GB)을 받는다.

끝나면 `export` 로 시작하는 두 줄이 나온다. 그 두 줄을 넣어야 한다.

```
$ echo 'export PATH="$HOME/whisper:$PATH"' >> ~/.bashrc
```

```
$ echo 'export WHISPER_MODEL="$HOME/whisper/ggml-large-v3.bin"' >> ~/.bashrc
```

Git Bash를 완전히 닫고 새로 연다

안 그러면 whisper가 안 잡힌다.

자동 다운로드가 실패하면 직접 받아도 된다

whisper.cpp 릴리스에서 `whisper-bin-x64.zip` 을 받아 `C:\Users\내이름\whisper` 에 풀고, 같은 폴더에 모델을 `ggml-large-v3.bin` 이름으로 저장한다.

확인

```
$ cd ~/shorts-factory
```

```
$ bash scripts/shorts doctor
```

이렇게 나와야 한다.

```
=== Tier 0 toolchain (Windows) ===
OK    ffmpeg      C:\Users\...\imageio_ffmpeg\binaries\ffmpeg-win-x86_64.exe
OK    yt-dlp       C:\...\python.exe -m yt_dlp
OK    whisper     C:\Users\...\whisper\whisper-cli.exe
OK    subtitles   자막을 입힐 수 있다 (libass)
OK    claude      C:\...\claude
OK    korean font 2 file(s) in ...\assets\fonts
OK    whisper model C:\Users\...\ggml-large-v3.bin
n/a   GEMINI_API_KEY not needed - whisper handles transcription
```

- **ffmpeg** 경로가 길고 이상해 보여도 정상이다. 파이썬 안에 들어 있는 것을 쓴다
- **yt-dlp** 가 **python.exe -m yt_dlp** 로 나오는 것도 정상이다
- **GEMINI_API_KEY** 가 **n/a** 인 것도 정상이다 — **whisper**가 있으면 안 쓴다
- **subtitles** 가 **MISS** 면 렌더가 안 된다 — 자막을 입히는 부품이 빠진 ffmpeg다. 5번을 다시 돌리면 바뀐다

마지막으로 실제 렌더가 되는지 본다. 인터넷 없이 30초, 무료.

```
$ bash scripts/smoke_test_render.sh
```

PASS 가 나오면 설치 끝이다.

명령은 전부 `bash scripts/shorts ...` 로 시작한다

맥이든 윈도우든 똑같다. 파이썬을 `python` 으로 부르지 `python3` 로 부를지는 이 스크립트가 알아서 고른다 — 그 차이 때문에 한쪽 명령을 다른 쪽에 붙여넣는 사고가 나서 없었다.

매번 하는 두 줄

Git Bash 를 열고:

```
$ cd ~/shorts-factory
```

```
$ git pull
```

플랜은 둘, 하나만 고른다

플랜 A

알아서 골라줘

채널의 주일예배 중 **아직 안 만든 편**을 무작위로 하나 뽑는다. 아무거나 하나 만들고 싶을 때.

```
$ bash scripts/shorts-auto.sh
```

이게 전부다. 인자를 붙이지 않는다.

플랜 B

이 설교로 해줘

유튜브에서 주소를 복사해 넣는다. 이번 주 설교를 꼭 찍어서 만들 때.

```
$ bash scripts/shorts-url.sh "주소"
```

주소를 **큰따옴표**로 감싼다. 유튜브 주소의 `?` 와 `&` 는 따옴표가 없으면 셸이 먼저 먹어버린다.

플랜 B — 주소는 어디서 복사하나

유튜브 **방배동 예심교회** 채널 → **실시간 스트림** 탭. 주일예배는 “동영상” 탭이 아니라 여기 있다. 동영상 탭은 수요예배와 새벽기도다.

폴더 이름은 그 **예배 날짜**로 자동으로 붙는다. 오늘 날짜가 아니다.

플랜 A — 뭐가 남았는지 먼저 보고 싶으면

```
$ bash scripts/shorts sermons
```

이미 만든 편은 숨겨서 보여준다. 목록만 읽는 거라 **돈이 안 든다**.

렌더 전에 화면 확인 · 1분

세로 화면은 가로 영상의 **3분의 1**만 쓴다. 설교자가 그 안에 들어왔는지 **굽기 전에** 사진으로 본다. 렌더는 10분 넘게 걸리지만 이건 1분이다.

```
$ bash scripts/shorts preview SUN-2026-08-09
```

preview/

├ clip-01.jpg 실제로 나올 세로 화면
└ clip-01-전체화면.jpg 원본 전체 + 노란 네모가 잘라낼 구간

파일 탐색기가 열리면 두 장을 본다. **-전체화면** 쪽의 노란 네모가 설교자를 감싸고 있으면 그대로 렌더한다. 네모가 빈 강단이나 화면 끝을 잡고 있으면 아래 **화면이 이상하게 잘렸을 때**로 간다.

새벽기도처럼 정지 이미지에 음성만 있는 영상은 노란 네모 없이 **화면 전체**가 들어간다. 그게 정상이다.

기다리기

막대 길이가 실제로 걸리는 시간의 비율이다. **전체 40분~1시간 반**. 맥의 M1/M2 칩과 달리 윈도우 PC는 그래픽카드 없이 CPU로만 도는 경우가 많아 전사가 더 걸릴 수 있다. 2단계에서 몇 분간 글자가 안 나와도 정상이다. 멈춘 게 아니다.

1/4 원본 내려받기		5-15분
2/4 한국어 전사		20-60분
3/4 구간 선별		2-5분
4/4 렌더		5-15분

- Git Bash 창을 **닫지 말 것**. 닫으면 중단된다
- 컴퓨터가 잠들면 멈춘다. 설정 → 시스템 → 전원 → 절전을 **안 함** 으로 두거나, 가끔 마우스를 움직인다
- 중간에 끊겨도 **같은 명령을 다시 치면** 이어서 간다. 받아둔 영상을 다시 받지 않는다

결과 보기

렌더가 끝나면 **파일 탐색기가 저절로 열린다**. 폴더 안이 이렇게 되어 있다.

SUN-2026-08-09/

├ renders/ ← MP4 3편. 여기서 확인한다
├ captions/ ← 자막이 틀리면 here를 고친다
├ publish-package.md ← 제목 · 설명문 · 선정 근거
└ source/ ← 받아둔 원본 (지워도 된다)

`renders` 안의 MP4 3개를 더블클릭해서 본다. 자막이 틀렸으면 한 단계 올라와 `captions` 로 들어가면 된다 — 이미 만들어져 있다.

창이 안 열렸으면:

```
$ bash scripts/shorts open SUN-2026-08-09
```

반대로 창이 자동으로 뜨는 게 거슬리면 명령 뒤에 `--no-open` 을 붙인다.

```
$ bash scripts/shorts render SUN-2026-08-09 --no-open
```

늘 안 뜨게 하려면 한 번만:

```
$ echo 'export SHORTS_NO_OPEN=1' >> ~/.bashrc
```

```
$ notepad office/production/SUN-2026-08-09/publish-package.md
```

각 클립의 제목 · 흑 · 설명문 · 왜 이 구간인지가 정리돼 있다. 유튜브에 올릴 때 그대로 쓰면 된다.

업로드는 사람이 한다

자동 업로드는 어느 단계에도 없다. 렌더가 마지막이다. `publish-package.md` 에 ⚠ 가 붙은 클립은 올리기 전에 확인한다 — 찬양이 깔렸거나 회중석 얼굴이 잡힌 경우다.

인스타그램 릴스 썸네일이 검게 나올 때

핸드폰 앱으로 올리면 괜찮는데 컴퓨터로 올리면 커버 후보 중 하나가 가끔 검은 프레임으로 잘못 뽑힌다. 영상 파일은 멀쩡하다 — 인스타그램이 컴퓨터 업로드를 처리하는 방식의 문제다.

-cover.jpg 를 직접 올린다

렌더할 때 `renders/` 안에 클립마다 `clip-01-cover.jpg` 같은 파일이 자동으로 같이 생긴다. 업로드 화면의 "커버 사진"에서 기본 후보가 검게 보이면 그대로 두지 말고 "컴퓨터에서 선택"을 눌러 이 파일을 직접 올린다. 영상 안에서 프레임을 끊어 고르는 것보다 게시 후에도 훨씬 안정적으로 유지된다.

방금 올렸는데 프로필 화면(격자)에서만 잠깐 검게 보이는 건 다른 문제다 — 게시물을 직접 열어보면 정상인데 격자 썸네일만 그렇다면, 인스타그램이 격자용 썸네일을 몇 분 늦게 만드는 것뿐이다. 새로고침하면 곧 정상으로 바뀐다.

폴더 이름(`SUN-2026-08-09`)은 본인 것으로 바뀌서 친다.

자막 글자가 틀렸을 때

이게 제일 흔하다. 음성 인식이 사람 이름이나 교회 이름을 잘못 듣는다.

한 단어가 계속 틀리는 경우

```
$ bash scripts/shorts fix SUN-2026-08-09 혜성교회 예심교회 --remember
```

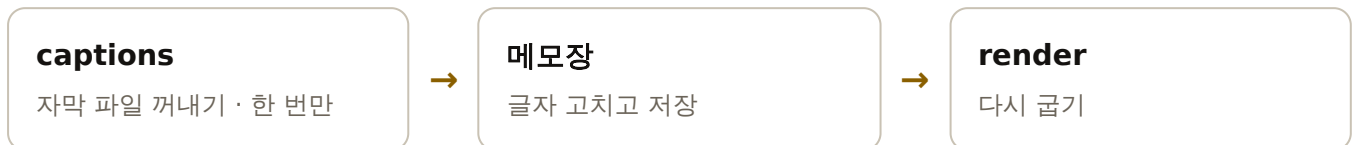
```
$ bash scripts/shorts render SUN-2026-08-09
```

전사본과 자막에서 그 말을 한 번에 바꾼다. 어디를 고쳤는지 시각과 함께 보여준다.

`--remember` 를 붙이면 앞으로 만드는 모든 편에 자동으로 적용된다. 교회 이름처럼 매번 틀리는 말은 한 번만 등록하면 된다.

문장을 직접 다듬고 싶은 경우

세 단계다. 첫 단계는 한 번만 하면 되고, 그다음은 고칠 때마다 반복한다.



1 · 자막 파일은 이미 있다

렌더가 끝나면 `captions` 폴더가 저절로 만들어져 있다. 따로 명령을 칠 필요가 없다.

```
captions/
├─ clip-01.srt
├─ clip-02.srt
├─ clip-03.srt
└─ 여기서 자막을 고칩니다.txt ← 뭘 하면 되는지 적혀 있다
```

renders/subs/ 안의 srt 는 고쳐도 소용없다

이름이 같아서 헷갈리는데, 그쪽은 렌더할 때마다 새로 만들어지는 결과물이다. 고칠 곳은 `captions/` 다.

혹시 폴더가 없으면 (옛날에 만든 편이라면) 이걸로 꺼낸다:

```
$ bash scripts/shorts captions SUN-2026-08-09
```

2 · srt 파일 열어서 고치기

`.srt` 는 글자만 든 파일이다. 메모장 종류면 다 열린다.

쓴다 메모장 — 컴퓨터에 기본으로 있다. VS Code 같은 편집기가 있으면 그것도 좋다

안 쓴다 워드 · 한글(HWP) · 엑셀 — 열리기는 하지만 저장할 때 자기 형식으로 바꿔 버려 파일이 망가진다

터미널에서 바로 열기

```
$ notepad office/production/SUN-2026-08-09/captions/clip-01.srt
```

파일 탐색기에서 열기

가장 쉬운 길 — 터미널에서 한 줄이면 그 폴더가 파일 탐색기로 열린다:

```
$ bash scripts/shorts open SUN-2026-08-09 captions
```

또는 파일 탐색기 창에서 `Ctrl + L` 를 누르고 주소를 붙여넣어도 된다:

```
%USERPROFILE%\shorts-factory\office\production\SUN-2026-08-09\captions
```

폴더가 열리면 `clip-01.srt` 를 오른쪽 버튼 클릭 → **연결 프로그램** → **메모장**.

더블클릭하면 윈도우가 어떤 프로그램으로 열지 물어볼 수 있다. 오른쪽 버튼으로 열면 바로 열린다.

매번 오른쪽 클릭이 귀찮으면 — 오른쪽 버튼 → **속성** → **연결 프로그램** → **변경** → 메모장. 한 번만 해 두면 `.srt` 는 더블클릭으로 바로 열린다.

고치기

파일은 이렇게 생겼다.

```
1
00:00:00,000 --> 00:00:03,400
근데 그 강사님이 생각을 해 보니까
```

숫자와 --> 줄은 건드리지 않는다

시간은 그대로 두고 **맨 아랫줄 글자만** 고친다. 시간을 바꾸면 자막이 말과 어긋난다. 실수로 망가뜨리면 렌더가 멈추고 이유를 말해 준다.

고친 뒤 반드시 저장한다 — Ctrl + S

메모장은 저장하기 전까지 고친 내용을 **창에만** 들고 있다. 저장을 안 하고 렌더하면 자막이 그대로다 — 가장 흔한 사고다.

저장되면 창 제목의 별표(*)가 사라진다. 그게 저장됐다는 표시다.

메모장이 인코딩을 물으면 UTF-8

한글이 옛날 방식(CP949)으로 저장돼도 이제는 알아서 읽는다. 그래도 글자가 깨진 파일이면 **렌더를 거부하고** 무엇을 고치라고 알려준다 — 깨진 자막이 구워진 영상이 나오는 것보다 낫다.

저장이 됐는지 확인하려면 — 지금 파일에 뭐라고 적혀 있는지 그대로 읽어 준다:

```
$ bash scripts/shorts captions SUN-2026-08-09 --show
```

렌더할 때도 수정본을 쓰면 첫 줄을 찍어 준다. 그 줄이 내가 고친 대로면 제대로 들어간 것이다.

3. 다시 렌더

```
$ bash scripts/shorts render SUN-2026-08-09
```

`captions/` 에 파일이 있으면 렌더가 **그것을 쓴다**. 전사본은 쳐다보지도 않는다. 다시 전사해도 고친 자막은 그대로 남는다.

구간을 옮기면 자막도 새로 맞춘다

`clips.json` 의 시작·끝을 바꾸면 고쳐 둔 자막의 시간이 안 맞게 된다. 그때는 새 구간에 맞춰 다시 만들고, 고쳤던 파일은 `clip-01.srt.이전` 으로 남긴다 — 글자만 옮겨 오면 된다.

한 편만 다시 만들려면 — 3편 전부 다시 구울 필요가 없다:

```
$ bash scripts/shorts render SUN-2026-08-09 --only clip-02
```

다운로드와 전사는 건너뛰고 굽기만 하므로 **클립 하나에 1~3분**이다.

고치기 전으로 되돌리고 싶으면 그 `srt`를 지우고 `captions` 를 다시 돌린다.

자막이 영상과 안 맞을 때

글자는 맞는데 나오는 때가 어긋나는 경우다. 말이 끝난 뒤에 자막이 뜨거나, 미리 떠 버린다. 글자가 틀린 것과는 원인이 다르다.

글자가 틀린 것과 때가 어긋난 것은 다르다

글자가 틀렸으면 `fix` 나 자막 파일 수정이다. 때가 어긋났으면 그 자막 파일이 다른 구간에 맞춰 만들어진 것이다 — 구간을 바꾼 뒤 옛 자막이 남아 있을 때 생긴다.

확인만 한 줄이면 된다. 전사본과 대조해 몇 초 어긋났는지 말해 준다.

```
$ bash scripts/shorts captions SUN-2026-08-09 --show
```

```
clip-01
  x 25초 늦게 나온다 - 이 자막은 다른 구간에 맞춰진 파일이다
clip-02
  영상과 맞는다 (어긋남 +0.1초, 일치도 96%)
```

어긋났다고 나오면 그 자막을 지우고 지금 구간에 맞춰 다시 꺼낸다.

```
$ rm office/production/SUN-2026-08-09/captions/clip-01.srt
```

```
$ bash scripts/shorts captions SUN-2026-08-09
```

```
$ bash scripts/shorts render SUN-2026-08-09 --only clip-01
```

고쳐 둔 글자가 아까우면 `.이전` 파일에서 옮겨 온다 — 지우지 않고 남겨 둔다.

구간이 마음에 안 들 때

초를 직접 조정한다

```
$ notepad office/production/SUN-2026-08-09/clips.json
```

```
$ bash scripts/shorts render SUN-2026-08-09
```

아예 다시 골라달라고 한다

```
$ bash scripts/shorts select SUN-2026-08-09 --count 3
```

```
$ bash scripts/shorts render SUN-2026-08-09
```

설교 구간을 잘못 잡았다

찬양이나 광고가 클립에 들어갔다면 설교 시작·끝 시각을 직접 준다. 전사부터 다시 하므로 시간이 걸린다.

```
$ rm office/production/SUN-2026-08-09/transcript.json
```

```
$ bash scripts/shorts transcribe SUN-2026-08-09 --sermon-window 0:45:00 1:20:00
```

화면이 이상하게 잘렸을 때

설교자가 한쪽으로 치우쳐 잘렸다면, 그 영상의 화면 배치가 평소와 달랐던 것이다. 예심교회 방송은 **해마다 배치가 바뀌었고** 강단 위치도 주일마다 달라진다.

설교자를 찾아서 가운데 놓는다

고정된 좌표를 쓰지 않는다. 1초 간격 프레임을 설교 구간 여덟 지점에서 비교해 **움직이는 사람**을 찾고 그를 가운데 둔다 — 배경 · 자막 · 그래픽은 움직이지 않으니 구분된다. 새벽기도처럼 **정지 이미지에 음성만** 있는 영상은 사람이 없으므로 자르지 않고 화면 전체를 9:16에 맞춘다.

먼저 `preview` 로 지금 어디를 잡고 있는지 사진으로 확인한다. 렌더를 다시 돌리지 않아도 된다.

```
$ bash scripts/shorts preview SUN-2026-08-09
```

그래도 어긋났으면 `clips.json` 의 `crop` 을 직접 준다. 숫자는 원본 영상 기준 픽셀이다.

```
"crop": { "x": 126, "y": 108, "h": 252 }
```

x 왼쪽에서 몇 픽셀부터
y 위에서 몇 픽셀부터
h 높이 (꼭은 9:16에 맞춰 자동)

```
$ bash scripts/shorts render SUN-2026-08-09
```

간단히 `"crop": "center"` · `"left"` · `"right"` 로 써도 되고, 정지 이미지처럼 **자르지 않고 통째로** 넣으려면 `"crop": "fit"` 이다.

엔드카드

영상 끝에 4초 붙는 카드다. **날짜 · 본문 · 제목 · 설교자 · 교회명**이 들어간다. 쇼츠는 채널을 떠나 혼자 돌아다니기 때문에, 이걸 보고 유튜브에 검색하면 원래 설교 영상이 나온다.

2024년 6월 23일 주일예배

신명기 12:1-8

택하신 곳으로
나아가야 하는 이유

설교 · 장선기 목사

방배동 예심교회

youtube.com/@yeshim1126

- 자동 유튜브 영상 제목에서 다섯 가지를 뽑아 저절로 만든다
- 본문 풀어서 쓴다 — 삼하 → 사무엘하. 그래야 검색된다
- 날짜 제목이 아니라 폴더 이름에서 가져온다

내용을 고치거나 빼려면

```
$ notepad office/production/SUN-2026-08-09/end-card.json
```

```
$ bash scripts/shorts render SUN-2026-08-09
```

아예 빼고 싶으면:

```
$ bash scripts/shorts render SUN-2026-08-09 --no-end-card
```

용량이 찰 때 — 원본만 지운다

한 편이 **300MB**쯤 되는데 그 **90%** 이상이 받아둔 원본 영상이다. 완성된 MP4와 자막은 다 합쳐도 20MB 남짓이다.

폴더	크기	지워도 되나
source/	약 275MB	지운다 — 다시 받을 수 있다
renders/	약 21MB	남긴다 — 완성된 MP4
captions/ 등 나머지	0.1MB	남긴다 — 자막 · 전사본 · 선정 근거

파일 탐색기에서 **source** 폴더만 휴지통에 넣으면 된다. 폴더 자체는 남겨 둔다 — 어떤 설교를 만들었는지 남고, 다시 뽑히지도 않는다.

SUN-2026-08-09/

├ renders/ 남긴다
├ captions/ 남긴다
├ source/ ← 이것만 지운다
├ transcript.json 남긴다
├ clips.json 남긴다
└ meta.json 남긴다 - 어떤 영상이었는지

폴더를 통째로 지워도 다시 뽑히지 않는다

만든 기록은 폴더 밖 `office/produced.json` 에도 따로 남는다. 실수로 폴더 안을 다 비워도, 폴더째 지워도 그 설교는 목록에서 계속 빠진다. 실제로 지워 보고 확인한 동작이다.

터미널이 편하면:

```
$ rm -rf office/production/SUN-2026-08-09/source
```

원본이 다시 필요하면 같은 주소로 받으면 된다. 기록이 있어도 **주소를 직접 주면 받는다** — 막지 않는다. 전사본과 자막이 남아 있으면 전사도 다시 하지 않는다.

막힐 때

화면에 이게 나오면

나온 것	뜻	할 것
python: command not found	PATH 체크 안 함	Python 재설치, Add python.exe to PATH 체크
자막을 고쳤는데 그대로	저장이 안 됨 / 다른 폴더	Ctrl+S 후 captions --show 로 확인
bash: git: command not found	Git 미설치	1부 1번
npm: command not found	Node 미설치 / 창 안 닫음	설치 후 Git Bash 새로 열기
no such file or directory	폴더가 아님	cd ~/shorts-factory
URL이 아니다: '#'	설명까지 복사됨	회색 상자 안만 복사

나온 것	뜻	할 것
인자를 받지 않는다	위와 같음	명령만 친다
전사할 수단이 없다	whisper 미설치	1부 5번
No such filter: 'subtitles'	자막 부품 없는 ffmpeg	setup_render_env.sh 를 다시 돌린다
GEMINI_API_KEY not set	옛날 코드	git pull
자막이 네모(□□□)로 나옴	폰트 문제	assets/fonts 에 ttf 2개 확인
30분간 아무것도 안 나옴	전사 중	정상. 기다린다
릴스 썸네일이 검게 나옴	컴퓨터 업로드 버그	renders/ 의 -cover.jpg 를 "컴퓨터에서 선택"으로 직접 올린다

그래도 안 되면 **Git Bash** 에 나온 글자를 그대로 복사해서 Claude에게 붙여넣는다. 어디서 멈췄는지가 그 안에 다 있다.

윈도우에서만 나오는 것들

- 백신 프로그램이 `whisper-cli.exe` 를 막는 경우가 있다. 차단 알림이 뜨면 허용한다
- 한글을 Git Bash에 붙여넣는 게 잘 안 되면, 메모장에 명령을 쓴 다음 복사해 붙여넣으면 된다
- 메모장으로 자막을 저장할 때 인코딩을 물으면 **UTF-8** 로 저장한다
- 전사가 아주 느리면 NVIDIA 그래픽카드가 있는 PC가 낫다. 그래도 CPU로 끝까지 돌아가긴 한다

참고

만들어지는 것의 규칙

- 9:16 세로, 1080×1920, **1.5배속**
- 자막 **노란색**, 유튜브 UI(`@yeshim1126`)와 안 겹치게 올려 둬
- 끝에 **엔드카드 4초** — 날짜 · 본문 · 제목 · 설교자 · 교회명
- 구간은 **설교 안에서만** 자른다. 찬양 · 기도 · 광고는 애초에 제외
- 목사님 음성은 **실제 하신 말씀 그대로**. 합성하지 않는다
- 15초 미만은 버린다. 왜 그 구간인지 근거가 없으면 렌더를 거부한다
- **업로드는 사람이 한다**. 렌더가 마지막 단계다
- 렌더가 끝나면 폴더 창이 저절로 열린다 — 싫으면 `--no-open`

저장소 `shorts-factory` · 브랜치 `claude/shorts-creation-automation-0xaii8`

같은 내용이 `docs/설치-윈도우.md` 에도 들어 있다.